

Text Classification

NLP - Week#05

Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' theorem with the assumption of independence between features. Despite its simplicity, Naive Bayes has been widely used in various applications, including spam filtering, sentiment analysis, and document classification.

The "naive" part refers to the algorithm's assumption that every word in the document is independent of the others, which rarely holds in reality, but simplifies calculations considerably and often gives surprisingly good results for text data

Probability: Basics

- Imagine you have a corpus of 20 tweets that can be categorized as having either positive or negative sentiment, but not both.
- Within that corpus, the word happy is sometimes being labeled positive and in some cases, negative.

Corpus of tweets

		Positive		
		Negative		

Tweets containing the word "happy"

		Positive		
			“happy”	
		Negative		

Probability: Basics

One way to think about probabilities is by counting how frequently events occur.

Let's define event A as a tweet being labeled positive.

Corpus of tweets

		Positive		
		Negative		

$A \rightarrow$ Positive tweet

$$P(A) = N_{\text{pos}} / N = 13 / 20 = 0.65$$

$$P(\text{Negative}) = 1 - P(\text{Positive}) = 0.35$$

Probability: Basics

Next, let's define event B in a similar way by counting tweets containing the word happy.

Tweets containing the word
"happy"

$B \rightarrow$ tweet contains "happy".

$$P(B) = P(\text{happy}) = N_{\text{happy}} / N$$

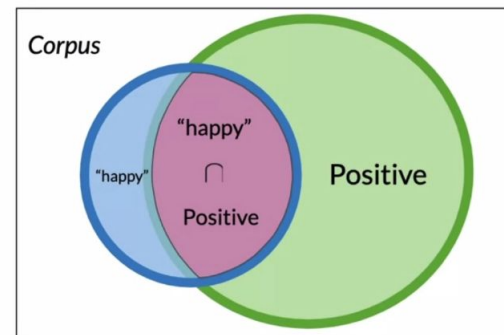
$$P(B) = 4 / 20 = 0.2$$

Probability: Basics

- The diagram shows 3 tweets labeled positive that also contain the word “happy,” represented by the overlapping pink area.
- The Venn diagram illustrates this as the intersection between tweets labeled positive and tweets containing the word “happy.”



$$P(A \cap B) = P(A, B) = \frac{3}{20} = 0.15$$



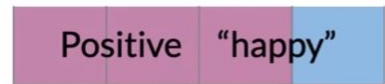
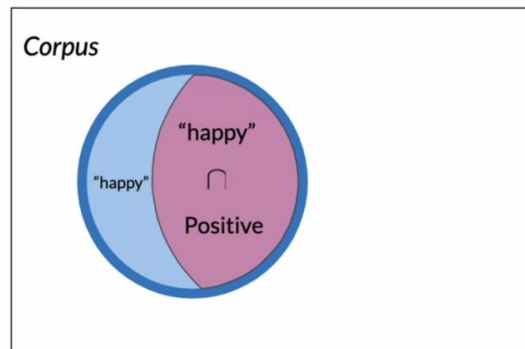
$$\begin{aligned} P(\text{tweet is labeled positive and contains "happy"}) &= \frac{P(\text{intersection})}{P(\text{entire corpus})} = \frac{P(\text{"happy"} \cap \text{positive})}{P(\text{"happy"} \cup \text{positive})} \\ &= \frac{3}{20} = 0.15 \text{ or } 15\%. \end{aligned}$$

Conditional Probability

Conditional Probability means the probability of an event happening given that another event has already happened.

In this scenario, the probability that a tweet is positive given that it contains the word happy, denoted by $P(\text{positive} \mid \text{"happy"})$, simply becomes:

$$\begin{aligned} P(\text{positive} \mid \text{"happy"}) &= \frac{\text{number of positive tweets and contain "happy"}}{\text{number of tweets that contain "happy"}} \\ &= \frac{P(\text{positive} \cap \text{happy})}{P(\text{"happy"})} \end{aligned}$$



$$P(A \mid B) = P(\text{Positive} \mid \text{"happy"})$$

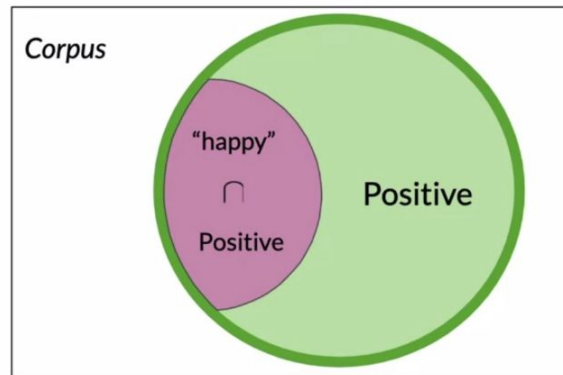
$$P(A \mid B) = 3 / 4 = 0.75$$

Conditional Probability

Now, let's find the probability that a positive tweet contains the word happy.

Formally, the probability $P(\text{"happy"} \mid \text{positive})$ is

$$\begin{aligned} P(\text{"happy"} \mid \text{positive}) &= \frac{\text{number of positive tweets that contain "happy"}}{\text{number of tweets that are positive}} \\ &= \frac{P(\text{"happy"} \cap \text{positive})}{P(\text{positive})} \end{aligned}$$

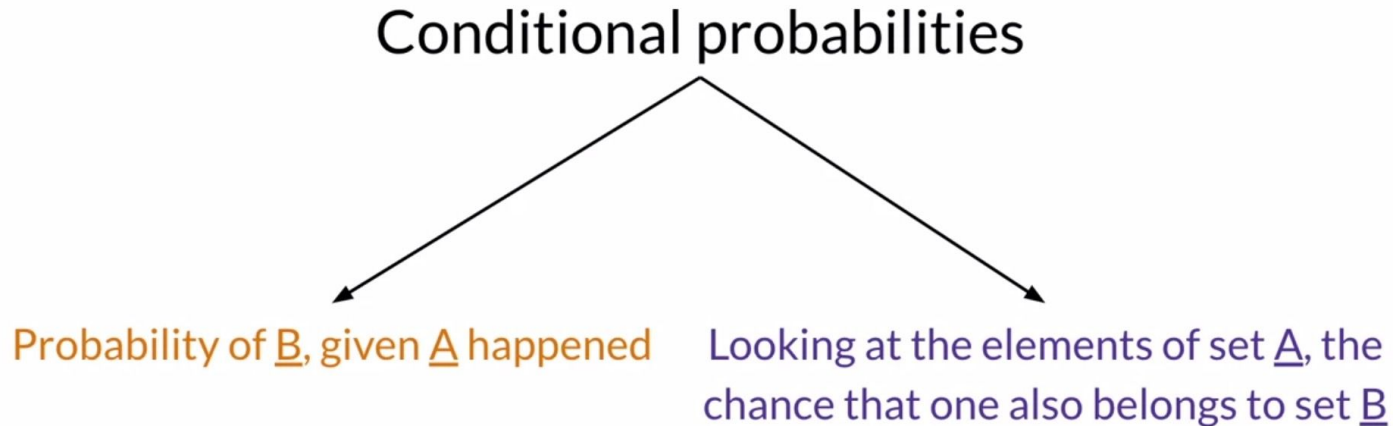


$$P(B \mid A) = P(\text{"happy"} \mid \text{Positive})$$

$$P(B \mid A) = 3 / 13 = 0.231$$

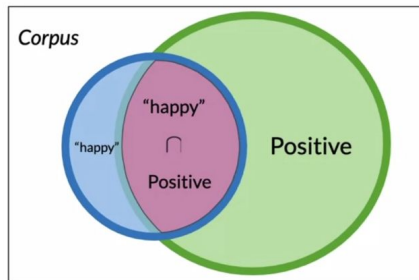
Conditional Probabilities

Conditional probabilities can be interpreted as the probability of an outcome B knowing that event A has already happened. In other words, given that we're looking at an element from set A, the probability that it also belongs to set B



Bayes' Rule

A graphical way to interpret this is using the Venn diagram shown below. Using the previous example as reference, the probability of a tweet being positive given that it has the word happy is:



$$P(\text{positive} \mid \text{"happy"}) = \frac{P(\text{positive} \cap \text{"happy"})}{P(\text{"happy"})}$$

$$P(\text{Positive} \mid \text{"happy"}) = \frac{P(\text{Positive} \cap \text{"happy"})}{P(\text{"happy"})}$$

Bayes' Rule connects the probability of X given Y with Y given X.

In our example:

Think about the probability of a tweet being positive given that it contains the word happy $P(\text{positive} \mid \text{"happy"})$ in terms of the probability of a tweet containing the word happy given that it is positive $P(\text{"happy"} \mid \text{positive})$, with a little algebraic manipulation:

$$P(\text{positive} \mid \text{"happy"}) = P(\text{"happy"} \mid \text{positive}) \times \frac{P(\text{positive})}{P(\text{"happy"})}$$

$$P(X \mid Y) = P(Y \mid X) \times \frac{P(X)}{P(Y)}$$

Bayes's Rule - Example

As an example, suppose that in your dataset, 25% of the positive tweets contain the word 'happy', 13% of the tweets in your dataset contain the word happy, and 40% of the total number of tweets are positive. You observe the tweet: 'happy to learn NLP'. The probability that this tweet is positive is:

$$P(\text{positive} \mid \text{"happy"}) = P(\text{"happy"} \mid \text{positive}) \times \frac{P(\text{positive})}{P(\text{"happy"})} = \frac{25}{100} \times \frac{40}{13} = 0.77 \text{ or } 77\%.$$

Example

Positive tweets

I am happy because I am learning NLP
I am happy, not sad.

Negative tweets

I am sad, I am not learning NLP
I am sad, not happy

Computing the Conditional Probabilities Table

1. **Generate positive/negative frequency features:** Extract the unique words (vocabulary) from both positive and negative corpora and record how many times each word appears in each corpus.
2. **Fetch total counts:** Calculate the total number of words in the positive and negative corpora by summing all word counts; in this example, each corpus contains 13 words in total.
3. **Compute conditional probabilities:** For each word, calculate the probability of it appearing given a class by dividing its frequency in that class by the total number of words in that class (e.g., for “I” in the positive class: $3/13=0.243/13 = 0.243/13=0.24$). Repeat for all words to form a conditional probability table for both positive and negative classes.
4. **Sanity check:** The sum of all conditional probabilities within each class should equal 1 — confirming that the probabilities are correctly normalized.

1. **Interpretation of conditional probabilities:**

Words with similar probabilities in both classes (e.g., I, am, learning, NLP) are neutral and add little to sentiment prediction, while words like happy, sad, and not show large probability differences and strongly influence sentiment.

2. **Zero probability issue:** If a word appears only in one class (e.g., because), its probability in the other class becomes 0, making comparison impossible — this is handled using Laplacian smoothing.

word	Pos	Neg
I	0.24	0.25
am	0.24	0.25
happy	0.15	0.08
because	0.08	0
learning	0.08	0.08
NLP	0.08	0.08
sad	0.08	0.17
not	0.08	0.17

Laplacian Smoothing

- Zero probability issue: If a word's probability for a class is 0, it causes problems in Naive Bayes calculations —
 - In the denominator (negative class): leads to a divide-by-zero error.
 - In the numerator (positive class): makes the entire inference expression zero out.
- This problem occurs when a word never appears in a class, and it's later resolved using Laplacian (additive) smoothing.

word	Pos	Neg
I	0.24	0.25
am	0.24	0.25
happy	0.15	0.08
because	0.08	0
learning	0.08	0.08
NLP	0.08	0.08
sad	0.08	0.17
not	0.08	0.17

Laplacian Smoothing

$$P(w_i | \text{class}) = \frac{\text{freq}(w_i, \text{class})}{N_{\text{class}}} \quad \text{class} \in \{\text{Positive}, \text{Negative}\}$$

$$P(w_i | \text{class}) = \frac{\text{freq}(w_i, \text{class}) + 1}{N_{\text{class}} + V}$$

N_{class} = frequency of all words in class

V = number of unique words in vocabulary

Laplacian Smoothing

word	Pos	Neg
I	3	3
am	3	3
happy	2	1
because	1	0
learning	1	1
NLP	1	1
sad	1	2
not	1	2
Nclass	13	12

$$P(I|Neg) = \frac{3 + 1}{12 + 8}$$

$$V = 8$$

word	Pos	Neg
I	0.19	0.20

Laplacian Smoothing

word	Pos	Neg
I	3	3
am	3	3
happy	2	1
because	1	0
learning	1	1
NLP	1	1
sad	1	2
not	1	2
Nclass	13	12

Laplacian Smoothing

$$V = 8$$

word	Pos	Neg
I	0.19	0.20
am	0.19	0.20
happy	0.14	0.10
because	0.10	0.05
learning	0.10	0.10
NLP	0.10	0.10
sad	0.10	0.15
not	0.10	0.15
Sum	1	1

Class Activity

Review	Class
"The movie was amazing!"	Positive
"I didn't like the film."	Negative
"Great acting and an engaging plot."	Positive
"Terrible movie, I would not recommend it to anyone."	Negative

Naive Bayes

Formally, the Naive Bayes inference condition rule for binary classification can be expressed as follows:

$$\prod_{i=1}^m \frac{P(w_i | \text{pos})}{P(w_i | \text{neg})}$$

Example

Tweet: I am happy today; I am learning.

$$\prod_{i=1}^m \frac{P(w_i|pos)}{P(w_i|neg)}$$

word	Pos	Neg
I	0.20	0.20
am	0.20	0.20
happy	0.14	0.10
because	0.10	0.05
learning	0.10	0.10
NLP	0.10	0.10
sad	0.10	0.15
not	0.10	0.15

Do this example by yourself

Solution

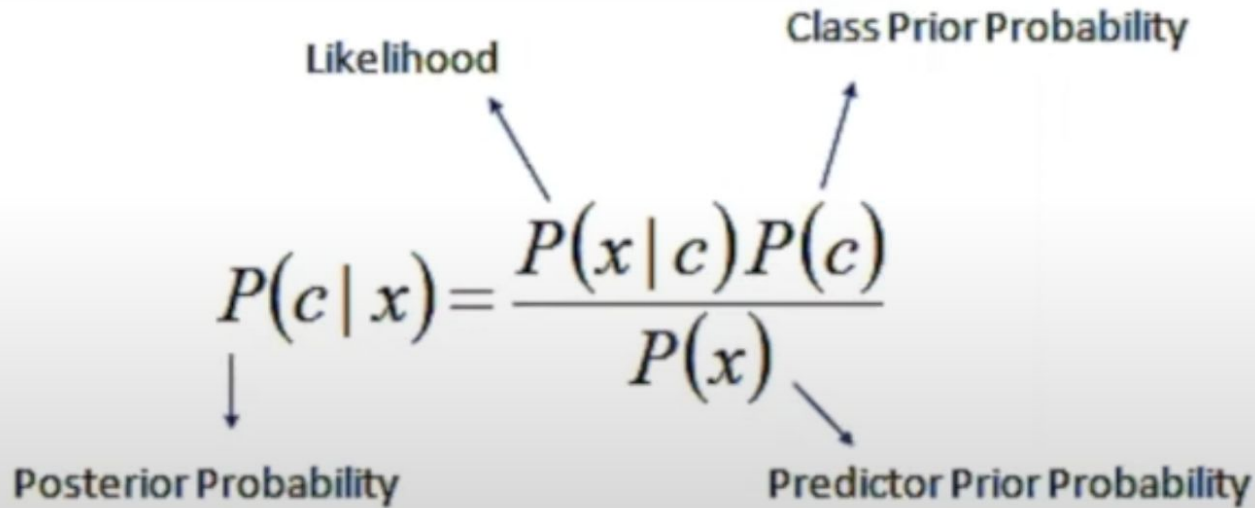
Tweet: I am happy today; I am learning.

$$\prod_{i=1}^m \frac{P(w_i|pos)}{P(w_i|neg)} = \frac{0.14}{0.10} = 1.4 > 1$$
$$\frac{\cancel{0.20}}{\cancel{0.20}} * \frac{\cancel{0.20}}{\cancel{0.20}} * \frac{0.14}{0.10} * \frac{\cancel{0.20}}{\cancel{0.20}} * \frac{\cancel{0.20}}{\cancel{0.20}} * \frac{\cancel{0.10}}{\cancel{0.10}}$$

word	Pos	Neg
I	0.20	0.20
am	0.20	0.20
happy	0.14	0.10
because	0.10	0.05
learning	0.10	0.10
NLP	0.10	0.10
sad	0.10	0.15
not	0.10	0.15

Since this value is higher than 1, we can infer that the words in the tweet collectively correspond to a positive sentiment, so you conclude that the tweet is positive.

Naive Bayes Classifier



The diagram shows the Naive Bayes formula with arrows pointing from descriptive labels to the corresponding parts of the equation:

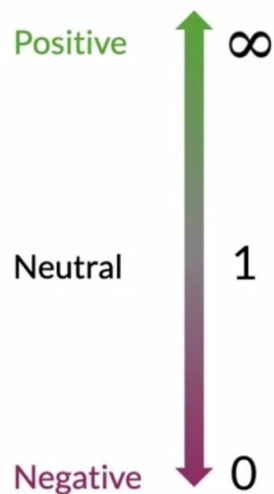
$$P(c | x) = \frac{P(x | c) P(c)}{P(x)}$$

Labels and their corresponding terms in the formula:

- Likelihood** points to $P(x | c)$
- Class Prior Probability** points to $P(c)$
- Posterior Probability** points to $P(c | x)$
- Predictor Prior Probability** points to $P(x)$

$$P(c | X) = P(x_1 | c) \times P(x_2 | c) \times \cdots \times P(x_n | c) \times P(c)$$

Log Likelihood



word	Pos	Neg	ratio
I	0.20	0.20	1
am	0.20	0.20	1
happy	0.14	0.10	1.4
because	0.10	0.10	1
learning	0.10	0.10	1
NLP	0.10	0.10	1
sad	0.10	0.15	0.6
not	0.10	0.15	0.6

$$\text{ratio}(w_i) = \frac{P(w_i | \text{Pos})}{P(w_i | \text{Neg})}$$

$$\approx \frac{\text{freq}(w_i, 1) + 1}{\text{freq}(w_i, 0) + 1}$$

Log Likelihood

Classification rule: A tweet is classified as positive if the product of these word ratios (likelihood) is greater than 1, and negative if it's less than 1 — this likelihood reflects how strongly the words favor one sentiment over the other.

class $\in \{pos, neg\}$

w $\rightarrow$ Set of *m* words in a tweet

$$\frac{P(pos)}{P(neg)} \prod_{i=1}^m \frac{P(w_i|pos)}{P(w_i|neg)} > 1$$

- A simple, fast, and powerful baseline
- A probabilistic model used for classification

Log Prior

The prior represents the ratio of class probabilities — how likely a tweet is positive vs. negative:

Note that $P(\text{pos})$ and $P(\text{neg})$ represent $P(D_{\text{pos}})$ and $P(D_{\text{neg}})$ in the above figure.

$$P(D_{\text{pos}}) = \frac{D_{\text{pos}}}{D}$$

$$P(D_{\text{neg}}) = \frac{D_{\text{neg}}}{D}$$

The log prior rescales this ratio to avoid large/small values:

$$\text{log prior} = \log\left(\frac{P(D_{\text{pos}})}{P(D_{\text{neg}})}\right) = \log\left(\frac{D_{\text{pos}}}{D_{\text{neg}}}\right)$$

It adjusts the likelihood for class imbalance, ensuring fair weighting between positive and negative tweets.

Preventing Numerical Underflow

Multiplying many small probabilities (values between 0 and 1) can cause underflow errors.

Using logarithms, we convert multiplication into addition:

$$\log(a \cdot b) = \log(a) + \log(b)$$

This stabilizes computation and ensures accurate results when calculating sentiment likelihoods.

Binary Classification Formulation

Log Likelihood

$$\frac{P(pos)}{P(neg)} \prod_{i=1}^m \frac{P(w_i|pos)}{P(w_i|neg)}$$

- Products bring risk of underflow

- $\log(a * b) = \log(a) + \log(b)$

- $\log\left(\frac{P(pos)}{P(neg)} \prod_{i=1}^n \frac{P(w_i|pos)}{P(w_i|neg)}\right) \Rightarrow \log \frac{P(pos)}{P(neg)} + \sum_{i=1}^n \log \frac{P(w_i|pos)}{P(w_i|neg)}$

log prior + log likelihood

Example

Now, let's use the method we've derived so far to classify the tweet, "I'm happy because I'm learning".

tweet: I am happy because I am learning.

$$\lambda(w) = \log \frac{P(w|pos)}{P(w|neg)}$$

$$\lambda(I) = \log \frac{0.05}{0.05} = \log(1) = 0$$

word	Pos	Neg	λ
I	0.05	0.05	0
am	0.04	0.04	
happy	0.09	0.01	
because	0.01	0.01	
learning	0.03	0.01	
NLP	0.02	0.02	
sad	0.01	0.09	
not	0.02	0.03	

doc: I am happy because I am learning.

$$\lambda(w) = \log \frac{P(w|pos)}{P(w|neg)}$$

$$\lambda(\text{happy}) = \log \frac{0.09}{0.01} \approx 2.2$$

word	Pos	Neg	λ
I	0.05	0.05	0
am	0.04	0.04	0
happy	0.09	0.01	2.2
because	0.01	0.01	
learning	0.03	0.01	
NLP	0.02	0.02	
sad	0.01	0.09	
not	0.02	0.03	

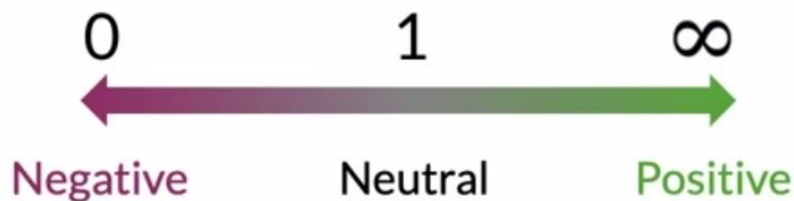
doc: I am happy because I am learning.

$$\sum_{i=1}^m \log \frac{P(w_i|pos)}{P(w_i|neg)} = \sum_{i=1}^m \lambda(w_i)$$

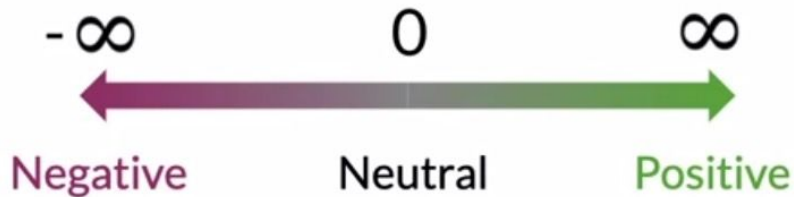
log likelihood = 0 + 0 + 2.2 + 0 + 0 + 0 + 1.1 = **3.3**

word	Pos	Neg	λ
I	0.05	0.05	0
am	0.04	0.04	0
happy	0.09	0.01	2.2
because	0.01	0.01	0
learning	0.03	0.01	1.1
NLP	0.02	0.02	0
sad	0.01	0.09	-2.2
not	0.02	0.03	-0.4

$$\prod_{i=1}^m \frac{P(w_i|pos)}{P(w_i|neg)} > 1$$



$$\sum_{i=1}^m \log \frac{P(w_i|pos)}{P(w_i|neg)} > 0$$



Training Naive Bayes

Step 0: Collect and annotate corpus

Positive tweets

I am happy because I am learning NLP
I am happy, not sad. @NLP

Negative tweets

I am sad, I am not learning NLP
I am sad, not happy!!

Step 1:
Preprocess

- Lowercase
- Remove punctuation, urls, names
- Remove stop words
- Stemming
- Tokenize sentences

Positive tweets

[happi, because, learn, NLP]
[happi, not, sad]

Negative tweets

[sad, not, learn, NLP]
[sad, not, happi]

Training Naive Bayes

Positive tweets

[happi, because, learn, NLP]

[happi, not, sad]

Negative tweets

[sad, not, learn, NLP]

[sad, not, happi]

Step 2:
Word
count

freq(w, class)

word	Pos	Neg
happi	2	1
because	1	0
learn	1	1
NLP	1	1
sad	1	2
not	1	2
N_{class}	7	7

freq(w, class)

word	Pos	Neg
happi	2	1
because	1	0
learn	1	1
NLP	1	1
sad	1	2
not	1	2
N_{class}	7	7

Step 3:
 $P(w|\text{class})$

$$V_{\text{class}} = 6$$

$$\frac{\text{freq}(w, \text{class}) + 1}{N_{\text{class}} + V_{\text{class}}}$$

word	Pos	Neg
happy	0.23	0.15
because	0.15	0.07
learning	0.08	0.08
NLP	0.08	0.08
sad	0.08	0.17
not	0.08	0.17

freq(w, class)

word	Pos	Neg
happi	2	1
because	1	0
learn	1	1
NLP	1	1
sad	1	2
not	1	2
N_{class}	7	7

Step 3:

$P(w|\text{class})$

$$V_{\text{class}} = 6$$

$$\frac{\text{freq}(w, \text{class}) + 1}{N_{\text{class}} + V_{\text{class}}}$$

$$\lambda(w) = \log \frac{P(w|\text{pos})}{P(w|\text{neg})}$$

Step 4: Get
lambda

word	Pos	Neg	λ
happy	0.23	0.15	0.43
because	0.15	0.07	0.6
learning	0.08	0.08	0
NLP	0.08	0.08	0
sad	0.08	0.17	-0.75
not	0.08	0.17	-0.75

Keys Takeaways

- Annotate a dataset with positive and negative labels for tweets.
- Pre-process the raw text to get a corpus of clean, standardized tokens.
- Compute the frequencies corresponding to each word and class (positive/negative) $\text{freq}(\mathbf{w}, \text{class})$.
- Compute the conditional probabilities corresponding to each word and class $P(\mathbf{w}|\text{pos}); P(\mathbf{w}|\text{neg})$ using the frequencies computed in the prior step, and applying the Laplacian smoothing formula.
- Obtain the conditional probability ratio $\frac{P(\mathbf{w}|\text{pos})}{P(\mathbf{w}|\text{neg})}$.
- Compute $\lambda(\mathbf{w})$ or the log-likelihood score for each word.
- Finally, estimate the log prior of the model or how **likely it is to see a positive tweet** in your corpus.

Testing Naive Bayes

Let's consider an example tweet, "I passed the NLP interview", and use our trained model to predict if this is a positive or negative tweet:

- log-likelihood dictionary $\lambda(w) = \log \frac{P(w|pos)}{P(w|neg)}$

- $logprior = \log \frac{D_{pos}}{D_{neg}} = 0$

- Tweet: [I, pass, the, NLP, interview] 

$$score = -0.01 + 0.5 - 0.01 + 0 + logprior = 0.48$$

$$pred = score > 0$$

word	λ
I	-0.01
the	-0.01
happi	0.63
because	0.01
pass	0.5
NLP	0
sad	-0.75
not	-0.75

Take Away

The Naive Bayes / Bag of Words approach is a powerful technique for text classification. It combines the simplicity of the BoW representation with the probabilistic reasoning of Naive Bayes to categorize text documents effectively. By treating words as independent features, Naive Bayes provides a computationally efficient and scalable solution for various NLP tasks.

Naive Bayes - Core Idea

Used to predict the probability that a document belongs to a specific class.

Each document is represented by its features (words).

Based on Bayes' Theorem:

$$P(\text{Class}|\text{Document}) = \frac{P(\text{Document}|\text{Class}) \times P(\text{Class})}{P(\text{Document})}$$

$P(\text{Class} | \text{Document}) \rightarrow$ Posterior probability (what we want to find)

$P(\text{Document} | \text{Class}) \rightarrow$ Likelihood (probability of seeing the words given the class)

$P(\text{Class}) \rightarrow$ Prior probability (how common the class is)

$P(\text{Document}) \rightarrow$ Evidence (overall probability of the document)

Chooses the class with the highest posterior probability as the prediction.

Applications

Author identification:

Author identification:

$$\frac{P(\text{👤} \mid \text{book})}{P(\text{👤} \mid \text{book})}$$

Application - Spam filtering

Using information taken from the sender, subject and content, we can decide whether an email is spam or not.

Spam filtering:

$$\frac{P(\text{spam}|\text{email})}{P(\text{nospam}|\text{email})}$$

Application - Information retrieval:

One of the earliest uses of Naive Bayes was filtering between relevant and irrelevant documents in a database.

$$P(\text{document}_k | \text{query}) \propto \prod_{i=0}^{|\text{query}|} P(\text{query}_i | \text{document}_k)$$

Retrieve document if $P(\text{document}_k | \text{query}) > \text{threshold}$

Application - Word disambiguation:

Word disambiguation involves breaking down words for contextual clarity.

$$\frac{P(\text{river}|\text{text})}{P(\text{money}|\text{text})}$$

Bank:



Assumptions

1. **Independence assumption:** Naive Bayes assumes that all features (words) are independent of each other given the class — meaning the presence of one word doesn't affect the presence of another.
2. **Model simplicity:** Called “naive” because of this strong assumption and the fact that it requires no custom parameters; however, violations of these assumptions (e.g., correlated words) can affect prediction accuracy.

Independence assumption

- **Independence example:** In the sentence “It is sunny and hot in the Sahara Desert,” Naive Bayes assumes each word occurs independently, ignoring relationships between them.
- **Reality check:** In practice, words like sunny and hot are correlated and contextually linked (e.g., to desert), so this assumption can lead to less accurate probability estimates.

“It is sunny and hot in the Sahara desert.”



Example

Let's consider the task of sentence completion, given a sentence: "It's always cold and snowy in ____", Naive Bayes might assign equal probability to the words spring, summer, fall, and winter even though from the context you can see that winter should be the most likely candidate.

In the coming classes, we will look into sophisticated methods that can handle these cases better.

"It's always cold and snowy in ____."



spring?? summer? fall?? winter??

Relative Class Frequencies in Corpus

Class distribution dependence: Naive Bayes performance depends on the class proportions in the training data — if the dataset is artificially balanced, it may not reflect real-world distributions.

Impact on results: In real tweet streams, positive tweets often dominate, while negative tweets are fewer (e.g., filtered or muted content). This mismatch can lead to a biased or skewed sentiment model.

Therefore - takeaways

The assumption of independence in Naive Bayes is very difficult to guarantee, but despite that, the model works pretty well in certain situations.

The relative frequency of positive and negative tweets in your training data sets needs to be balanced in order to deliver accurate results.

Error Analysis in Naive Bayes Sentiment Classification

1. Preprocessing issues:

- a. Removing punctuation or stop words can change the meaning

Tweet: My beloved grandmother ✕

processed_tweet: [belov, grandmoth]

- b. Dropping words like “not” can invert sentiment (e.g., “This is not good” → “good, attitude, nice” → falsely positive).

Tweet: This is not good, because your attitude is not even close to being nice.

processed_tweet: [good, attitude, close, nice]

Error Analysis in Naive Bayes Sentiment Classification

Word order sensitivity:

Naive Bayes ignores word order, missing context

Tweet: I am happy because I did not go.



Tweet: I am not happy because I did go.



Error Analysis in Naive Bayes Sentiment Classification

Adversarial attacks:

Struggles with sarcasm, irony, or euphemisms that humans easily understand.

Tweet: This is a ridiculously powerful movie. The plot was gripping and I cried right through until the ending!

processed_tweet: [ridicul, power, movi, plot, grip, cry, end]

Error Analysis - Key takeaways

- Errors in a Naive Bayes-powered system can happen at the following locations in a typical model's pipeline:
 - Preprocessing
 - Removing punctuation (example ':(')
 - Removing stop words
 - Word order
 - E.g.: I am happy because I did not go vs. I am not happy because I did go
 - Adversarial attacks
 - Easily detected by humans but algorithms are usually terrible at it!
 - Sarcasm, Irony, Euphemisms, etc.
 - E.g.: This is a ridiculously powerful movie. The plot was gripping and I cried right through until the ending.

Code Task

<https://colab.research.google.com/drive/1dKGHTtNhGb0KNBx19HozQ9MQzRzit77L?usp=sharing>

References

- <https://aman.ai/coursera-nlp/naive-bayes/>
- Chapter#02 of the book “Natural Language Processing by Jacob Eisenstein”
- https://scikit-learn.org/stable/modules/naive_bayes.html